

전자전기컴퓨터설계실험II

실험 전 보고서

LAB4: STM32_0 및 FPGA 조합회로

학과 전자전기컴퓨터공학부
이름/학번 김영서 / 2025440031
작성일 2026년 10월 12일
대상 STM32 LED·버튼, 외부 LED, 4비트 곱셈기, BCD→이진수 변환
검증 환경
FPGA 시뮬레이터: Icarus Verilog
FPGA Part: xc7s75fgga484-1

1. 실험 목적 및 공통 검증 조건

실험 목적

1) STM32 NUCLEO-G474RE의 GPIO 입력·출력과 HAL 함수를 이해하고, 사용자 버튼으로 LED 토글 간격을 1초, 2초, 4초, 8초 순서로 변경하는 비차단식 프로그램을 작성한다.

2) 보드 내장 LED와 PB0에 연결한 외부 LED에서 같은 제어 알고리즘을 확인하여, 소프트웨어 논리와 실제 GPIO 배선의 관계를 이해한다.

3) FPGA에서 4비트 무부호 곱셈기와 두 자리 BCD→이진수 변환기를 설계하고, self-checking 테스트벤치로 정상 입력·경계값·잘못된 BCD 입력을 검증한다.

공통 사전 검증 환경

구분	환경	사전 검증
STM32	NUCLEO-G474RE	GCC/CMake Debug 빌드
작성 도구	VS Code·STM32Cube	오류 0, exit code 0
FPGA	Icarus Verilog	VCD 기능 시뮬레이션
FPGA Part	xc7s75fgga484-1	합성 전 논리 확인
증거	코드·표·예상·실제 파형	사전 기능 검증 완료

표 1. 공통 개발 환경과 예비 검증 범위

예비보고서에서는 코드의 문법, 빌드 가능성, 설계식과 테스트벤치의 판정 조건을 먼저 확인한다. 보드 다운로드, 스위치·버튼 조작, 실제 LED 출력과 시연 영상은 본 실험에서 수행하고 결과보고서에 추가한다.

공통 판정 기준은 다음과 같다. STM32 코드는 지연 함수로 입력을 막지 않아야 하며, 버튼 한 번에 상태가 한 단계만 변해야 한다. FPGA 코드는 모든 정상 입력에서 계산값이 일치해야 하고, BCD 범위를 벗어난 입력은 valid=0으로 구분해야 한다.

2. STM32_0 · LED와 버튼

개발 환경과 프로젝트 구성

실습은 NUCLEO-G474RE와 수업용 ece2_474 프로젝트를 사용한다. main.c는 주변장치 초기화와 메인 루프를 제공하고, 학생 코드는 Core/Src/ece2_474_routine.c의 polling_routine()에 작성한다. ece2_474_init()은 초기화 뒤 한 번 호출되고 polling_routine()은 메인 루프에서 반복 호출된다.

신호	핀/매크로	역할
내장 LED	PA5 / LED_ece2	과제 1 출력
사용자 버튼	PC13 / BUTTON_ece2	간격 변경 입력
외부 출력 1	PB0 / GPIO_OUT1_ece2	과제 2 LED
외부 출력 2	PB1 / GPIO_OUT2_ece2	예비 출력
기준 전위	GND	외부 LED 귀환

표 2. STM32 실습 핀 구성

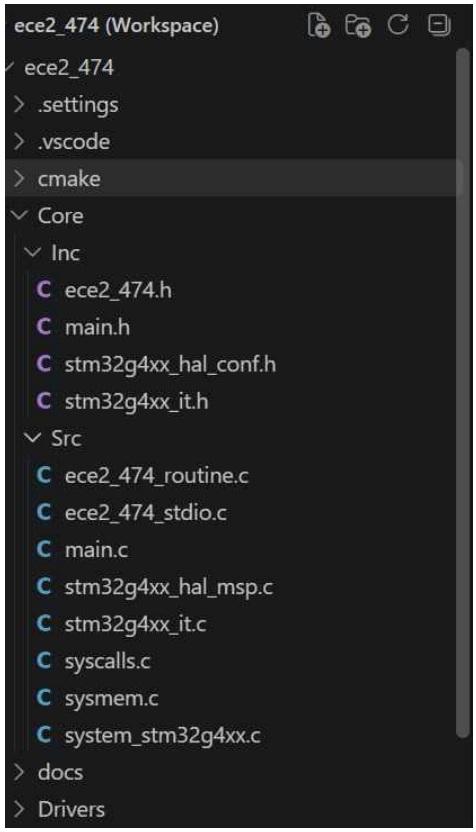


그림 1. VS Code에서 확인한 STM32 프로젝트 구성

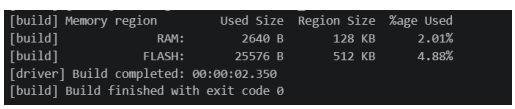


그림 2. CMake Debug 빌드 성공 결과

관련 이론

GPIO 입력과 출력

GPIO 입력은 버튼과 같은 외부 신호의 논리 상태를 읽고, GPIO 출력은 LED와 같은 부하에 HIGH 또는 LOW를 인가한다. 본 프로젝트에서는 HAL_GPIO_ReadPin()으로 PC13을 읽고 HAL_GPIO_TogglePin()으로 PA5 또는 PB0의 상태를 반전한다.

폴링과 비차단식 시간 처리

폴링은 메인 루프에서 입력을 반복 확인하는 방식이다. HAL_Delay()로 1~8초 동안 실행을 멈추면 그 사이의 버튼 입력을 놓칠 수 있으므로, HAL_GetTick()으로 현재 시간을 읽고 두 시각의 차이가 interval_ms 이상인지 비교한다. uint32_t 뺄셈을 사용하면 tick 값의 순환에도 일반적인 간격 비교가 유지된다.

에지 검출과 디바운싱

버튼을 계속 누른 상태에서 interval_ms가 반복 변경되지 않도록 previous_button과 current_button을 비교해 RESET→SET 변화만 새로운 입력으로 판정한다. 또한 마지막 유효 입력 뒤 200 ms 이내의 변화는 접점 바운스로 보고 무시한다. 눌림 논리는 현재 프로젝트 설정의 GPIO_PIN_SET을 기준으로 한다.

LED 토글 조건: now_ms − last_led_ms ≥ interval_ms

여기서 interval_ms는 LED 출력이 한 번 반전되는 간격이다. 따라서 LED가 ON에서 다시 ON이 되는 전체 점멸 주기는 2×interval_ms이다.

과제 1 · 버튼으로 내장 LED 간격 변경

초기 interval_ms는 1000 ms이다. 유효한 버튼 입력마다 값을 두 배로 만들고, 8000 ms 상태에서 다음 입력을 받으면 1000 ms로 되돌린다. 버튼 입력 직후 last_led_ms를 현재 시각으로 갱신하여 새 간격을 그 시점부터 적용한다.

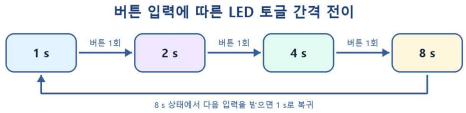


그림 3. 버튼 입력에 따른 LED 토글 간격 상태 전이

현재 상태	입력	interval	LED 동작	다음 상태
S0	없음	1000 ms	1초마다 반전	S0
S0	1회	2000 ms	2초마다 반전	S1
S1	1회	4000 ms	4초마다 반전	S2
S2	1회	8000 ms	8초마다 반전	S3
S3	1회	1000 ms	1초마다 반전	S0

표 3. 내장 LED 점멸 간격의 상태 전이

polling_routine() 구현

```
void polling_routine(void)
{
    static GPIO_PinState previous_button
= GPIO_PIN_RESET;
    static uint32_t last_press_ms = 0U;
    static uint32_t last_led_ms = 0U;
    static uint32_t interval_ms = 1000U;

    GPIO_PinState current_button =
HAL_GPIO_ReadPin(
        BUTTON_ece2_GPIO_Port,
BUTTON_ece2_Pin);
    uint32_t now_ms = HAL_GetTick();

    if (current_button == GPIO_PIN_SET
&&
        previous_button ==
GPIO_PIN_RESET &&
        now_ms - last_press_ms >=
200U)
    {
        last_press_ms = now_ms;
        interval_ms = (interval_ms >=
8000U)
                    ? 1000U :
interval_ms * 2U;
        last_led_ms = now_ms;
    }

    if (now_ms - last_led_ms >=
interval_ms)
    {
        last_led_ms = now_ms;
        HAL_GPIO_TogglePin(
            LED_ece2_GPIO_Port,
LED_ece2_Pin);
    }
    previous_button = current_button;
}
```

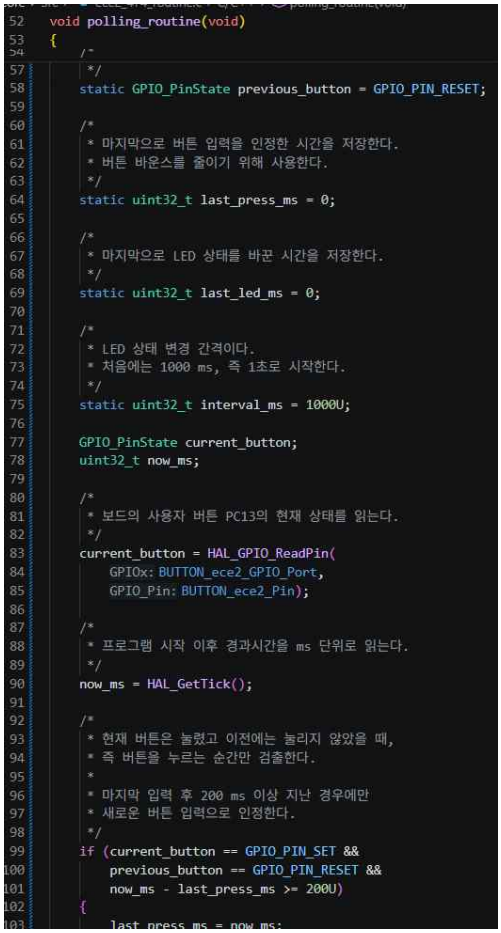


그림 4(a). 과제 1 코드 작성 화면 전반부

```
        {

            /*
             * 버튼을 누를 때마다 간격을 두 배로 증가시킨다.
             * 8초 상태에서 다시 누르면 1초로 돌아간다.
             */
            if (interval_ms >= 8000U)
            {
                interval_ms = 1000U;
            }
            else
            {
                interval_ms = interval_ms * 2U;
            }

            /*
             * 버튼을 누른 시점부터 변경된 시간을 다시 측정한다.
             */
            last_led_ms = now_ms;
        }

        /*
         * 설정된 시간이 지나면 내장 LED의 상태를 반전한다.
         */
        if (now_ms - last_led_ms >= interval_ms)
        {
            last_led_ms = now_ms;

            HAL_GPIO_TogglePin(
                GPIOx: LED_ece2_GPIO_Port,
                GPIO_Pin: LED_ece2_Pin);
        }

        /*
         * 현재 버튼 상태를 다음 반복의 이전 상태로 저장한다.
         */
        previous_button = current_button;
    }
```

그림 4(b). 과제 1 코드 작성 화면 후반부

확인 시점	버튼 입력	interval_ms	예상 출력
전원 직후	없음	1000	PA5 1초마다 반전
첫 입력	유효 1회	2000	2초 간격 적용
두 번째	유효 1회	4000	4초 간격 적용
세 번째	유효 1회	8000	8초 간격 적용
네 번째	유효 1회	1000	초기 간격 복귀

표 4. 과제 1의 단계별 예상 결과

과제 2 · 외부 LED 제어

PB0에서 330 Ω 전후의 전류 제한 저항과 LED를 거쳐 GND로 연결한다. LED의 긴 다리(애노드)는 저항을 통해 PB0에, 짧은 다리(캐소드)는 GND에 연결한다. 저항은 LED와 직렬이면 위치를 바꾸어도 전류 제한 기능이 같다.

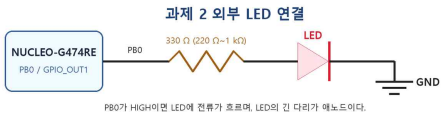


그림 5. PB0 외부 LED의 권장 연결

과제 2는 버튼 처리와 interval_ms 알고리즘을 그대로 사용하고 출력 핀만 PB0로 바꾼다. 아래 한 줄이 과제 1과의 핵심 차이이다.

```
HAL_GPIO_TogglePin(
    GPIO_OUT1_ece2_GPIO_Port,
    GPIO_OUT1_ece2_Pin);
```

따라서 전원 직후 외부 LED는 1초마다 반전하고, 버튼을 누를 때마다 2초, 4초, 8초, 1초 간격으로 순환해야 한다. LED가 켜지지 않으면 PB0 설정, LED 극성, 저항, 공통 GND를 먼저 확인한다.

예상 오류와 점검 방법

- 버튼 한 번에 여러 단계가 바뀌면 RESET→SET 에지 조건과 200 ms 디바운싱 조건을 확인한다.
- 버튼을 눌러도 변하지 않으면 BUTTON_ece2 핀 설정과 눌림 시 GPIO 값이 GPIO_PIN_SET인지 확인한다.
- LED가 일정하지 않으면 HAL_Delay()가 남아 있는지, last_led_ms와 interval_ms의 자료형이 uint32_t인지 확인한다.
- 외부 LED만 꺼져 있으면 GPIO_OUT1_ece2가 출력으로 설정되었는지와 PB0-저항-LED-GND 연결을 확인한다.

3. FPGA · 4비트 무부호 곱셈기

목적과 이론

두 4비트 무부호 입력 a와 b의 범위는 각각 0~15이다. 최대 곱은 15×15=225이므로 결과 m은 8비트가 필요하다.
조합회로이므로 클록이나 상태 저장 없이 입력이 변하면 전파 지연 뒤 결과가 바뀐다.

$$m[7:0] = a[3:0] \times b[3:0], \quad 0 \le m \le 225$$

파일	모듈	역할
multiply_unsigned.v	multiply_unsigned	폭 매개변수 곱셈기
mult_4bit.v	mult_4bit	4비트 최상위 래퍼
tb_design.v	tb	지정값·전수검사
simulation.json	시뮬레이션 설정	소스·최상위 지정
wave.png	파형 증거	VCD 기능 검증 캡처

표 5. 4비트 곱셈기 파일과 역할

설계 코드

```
`timescale 1ns/1ps
module mult_4bit(
    input  wire [3:0] a,
    input  wire [3:0] b,
    output wire [7:0] m
);
    multiply_unsigned #(.WIDTH(4))
u_multiply(
        .a(a), .b(b), .product(m));
endmodule

module multiply_unsigned #(parameter
WIDTH = 4)(
    input  wire [WIDTH-1:0] a,
    input  wire [WIDTH-1:0] b,
    output wire [2*WIDTH-1:0] product
);
    assign product = a * b;
endmodule
```



그림 6. 4비트 곱셈기 설계 코드 화면

Self-checking 테스트벤치

```
`timescale 1ns/1ps
module tb;
    reg  [3:0] a, b;
    wire [7:0] m;
    integer x, y;
    mult_4bit dut(.a(a), .b(b), .m(m));

    task check_key;
        input [3:0] aa, bb;
        begin
            a = aa; b = bb; #10;
            if (m !== aa * bb)
                $fatal(1, "multiply
mismatch");
        end
    endtask

    initial begin
        check_key(0,0);  check_key(3,5);
        check_key(7,9);
check_key(15,15);
        for (x=0; x<16; x=x+1)
            for (y=0; y<16; y=y+1) begin
                a=x; b=y; #1;
                if (m !== x*y)
                    $fatal(1,"a=%0d
b=%0d m=%0d",x,y,m);
            end
        $display("PASS multiplier: 256
input pairs");
        $finish;
    end
    initial begin #5000;
$fatal(1,"timeout"); end
endmodule
```

시간 구간	a	b	m	판정
0–10 ns	0	0	0	정상
10–20 ns	3	5	15	정상
20–30 ns	7	9	63	정상
30–40 ns	15	15	225	경계값
이후	0~15	0~15	a×b	256쌍 검사

표 6. 4비트 곱셈기의 핵심 구간 예상값



그림 7. 4비트 곱셈기의 핵심 구간 예상 파형

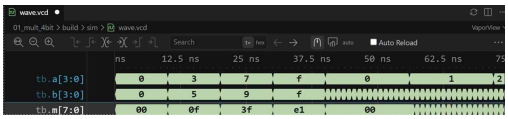


그림 8. 4비트 곱셈기 VCD 기능 시뮬레이션 파형

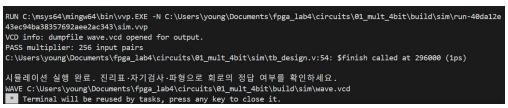


그림 9. 4비트 곱셈기 self-checking PASS 결과

시뮬레이션의 성공 조건은 오류 없이 PASS multiplier: 256 input pairs가 출력되는 것이다. m의 폭을 4비트로 선언하면 상위 비트가 잘려 15×15가 1로 보일 수 있으므로 반드시 8비트로 선언한다. signed형이 섞이지 않도록 모든 포트를 무부호 wire/reg로 둔다.

4. FPGA · BCD→이진수 변환기

목적과 이론

두 자리 BCD는 십의 자리 ten과 일의 자리 one을 각각 4비트로 표현한다. 각 자리의 유효 범위는 0~9이고, 이진수 출력은 0~99이므로 7비트가 필요하다. 두 자리 중 하나라도 10~15이면 정상 BCD가 아니므로 valid를 0으로 출력한다.

binary[6:0] = ten × 10 + one

valid = (ten ≤ 9) AND (one ≤ 9)

파일	모듈	역할
bcd_to_binary.v	bcd_to_binary	계산과 유효성 판정
bcd_conv.v	bcd_conv	최상위 래퍼
tb_design.v	tb	정상·비정상 전수검사
simulation.json	시뮬레이션 설정	소스·최상위 지정
wave.png	파형 증거	VCD 기능 검증 캡처

표 7. BCD 변환기 파일과 역할

설계 코드

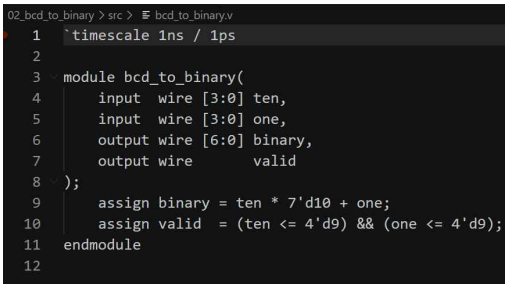
```
`timescale 1ns/1ps
module bcd_to_binary(
    input  wire [3:0] ten,
    input  wire [3:0] one,
    output wire [6:0] binary,
    output wire       valid
);
    assign binary = ten * 7'd10 + one;
    assign valid  = (ten <= 4'd9) &&
(one <= 4'd9);
endmodule

module bcd_conv(
    input  wire [3:0] one,
    input  wire [3:0] ten,
    output wire [6:0] bin,
    output wire       valid
);
    bcd_to_binary u_convert(
        .ten(ten), .one(one),
        .binary(bin), .valid(valid));
endmodule
```

02_bcd_to_binary > src > bcd_conv.v

```
1  `timescale 1ns / 1ps
2
3  module bcd_conv(
4      input  wire [3:0] one,
5      input  wire [3:0] ten,
6      output wire [6:0] bin,
7      output wire       valid
8  );
9      bcd_to_binary u_convert (
10         .ten(ten),
11         .one(one),
12         .binary(bin),
13         .valid(valid)
14     );
15 endmodule
16
```

그림 10(a). BCD→이진수 변환기 최상위 래퍼 bcd_conv.v



```
02_bcd_to_binary > src > bcd_to_binary.v
1   `timescale 1ns / 1ps
2
3   module bcd_to_binary(
4       input  wire [3:0] ten,
5       input  wire [3:0] one,
6       output wire [6:0] binary,
7       output wire       valid
8   );
9       assign binary = ten * 7'd10 + one;
10      assign valid  = (ten <= 4'd9) && (one <= 4'd9);
11  endmodule
12
```

그림 10(b). BCD→이진수 변환기 계산 및 유효성 판정 모듈 bcd_to_binary.v

Self-checking 테스트벤치

```
`timescale 1ns/1ps
module tb;
    reg  [3:0] ten, one;
    wire [6:0] bin;
    wire valid;
    integer t, o;
    bcd_conv dut(.one(one), .ten(ten),
                 .bin(bin), .valid(valid));

    task check_key;
        input [3:0] tt, oo;
        begin ten=tt; one=oo; #10; end
    endtask

    initial begin
        check_key(0,0); check_key(1,2);
        check_key(5,9); check_key(9,9);
        check_key(10,0);
        for (t=0; t<16; t=t+1)
            for (o=0; o<16; o=o+1)
begin
                ten=t; one=o; #1;
                if (valid !==
((t<10)&&(o<10)))
                    $fatal(1,"BCD valid
mismatch");
                if (t<10 && o<10 && bin
!== t*10+o)
                    $fatal(1,"BCD value
mismatch");
            end
        $display("PASS BCD: 100 valid,
156 invalid");
        $finish;
    end
    initial begin #5000;
$fatal(1,"timeout"); end
endmodule
```

시간 구간	ten	one	bin	valid
0–10 ns	0	0	0	1
10–20 ns	1	2	12	1
20–30 ns	5	9	59	1
30–40 ns	9	9	99	1
40–50 ns	10	0	무시	0

표 8. BCD→이진수 변환기의 핵심 구간 예상값



그림 11. BCD→이진수 변환기의 핵심 구간 예상 파형

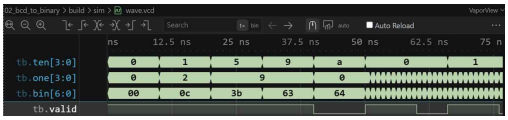


그림 12. BCD→이진수 변환기 VCD 기능 시뮬레이션 파형

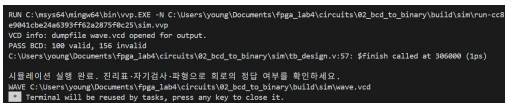


그림 13. BCD→이진수 변환기 self-checking PASS 결과

invalid 입력에서도 조합식 자체의 bin 값은 나타날 수 있으나 valid=0이므로 정상 데이터로 사용하지 않는다. 시뮬레이션의 성공 조건은 100개의 정상 조합과 156개의 비정상 조합을 모두 검사한 뒤 PASS BCD가 출력되는 것이다.
예상 오류는 ten과 one의 포트 순서를 바꾸는 경우, bin을 6비트로 선언해 99를 표현하지 못하는 경우, valid 조건을 OR로 작성하는 경우이다. 파형에서 ten, one, bin, valid를 함께 관찰하면 원인을 구분할 수 있다.

GitHub 저장소 및 재현 정보

vivado버전: vivado 2026.1

FPFA Part: xc7s75fgga484-1

LAB_04 github 주소:

dalkomys777/fpga_lab4: 전자전기컴퓨터설계실험II LAB4 - STM32_0 및 FPGA 조합회로

기준 브랜치: main

기준 커밋:

d6b7cc886f9771f58d9cd806d522c7403aed0aa9

LAB_04 제출 태그:

fpga_lab4-reports-pre

1번회로(4비트 곱셈기):

VS Code design code image:

fpga_lab4/evidence/01_mult_4bit/vscode/design_code.png at main · dalkomys777/fpga_lab4

VS Code simulation pass image:

fpga_lab4/evidence/01_mult_4bit/vscode/pass.png at main · dalkomys777/fpga_lab4

VS Code simulation log:

fpga_lab4/evidence/01_mult_4bit/vscode/simulation.log at main · dalkomys777/fpga_lab4

VS Code simulation text:

fpga_lab4/evidence/01_mult_4bit/vscode/simulation.txt at main · dalkomys777/fpga_lab4

VS Code waveform VCD:

fpga_lab4/evidence/01_mult_4bit/vscode/wave.vcd at main · dalkomys777/fpga_lab4

VS Code waveform image:

fpga_lab4/evidence/01_mult_4bit/vscode/wave.png at main · dalkomys777/fpga_lab4

VS Code waveform xdc:

fpga_lab4/circuits/01_mult_4bit/constraints/lab4_mult_4bit.xdc at main · dalkomys777/fpga_lab4

2번회로(bcd to binary):

VS Code design code image:

fpga_lab4/evidence/02_bcd_to_binary/vscode/design_code.png at main · dalkomys777/fpga_lab4

VS Code simulation pass image:

fpga_lab4/evidence/02_bcd_to_binary/vscode/pass.png at main · dalkomys777/fpga_lab4

VS Code simulation log:

fpga_lab4/evidence/02_bcd_to_binary/vscode/simulation.log at main · dalkomys777/fpga_lab4

VS Code simulation text:

fpga_lab4/evidence/02_bcd_to_binary/vscode/simulation.txt at main · dalkomys777/fpga_lab4

VS Code waveform VCD:

fpga_lab4/evidence/02_bcd_to_binary/vscode/wave.vcd at main · dalkomys777/fpga_lab4

VS Code waveform image:

fpga_lab4/evidence/02_bcd_to_binary/vscode/wave.png at main · dalkomys777/fpga_lab4

VS Code waveform xdc:

fpga_lab4/circuits/02_bcd_to_binary/constraints/lab4_bcd_to_binary.xdc at main · dalkomys777/fpga_lab4

전체 소스 커밋:

Commits · dalkomys777/fpga_lab4

6. 참고문헌

[1] Glaysia, ece2_474 course repository, README, STM32_0.md, FPGA_additional.md, GitHub, 접속일 2026-10-11, https://github.com/Glaysia/ece2_474
[2] STMicroelectronics, RM0440, STM32G4 Series advanced Arm-based 32-bit MCUs Reference Manual.
[3] STMicroelectronics, UM1724, STM32 Nucleo-64 Boards User Manual.
[4] Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013, ISBN 978-0-12-408082-9.
[5] AMD, Vivado Design Suite User Guide: Logic Simulation (UG900), https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation
[6] Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed., Pearson, 2003, ISBN 978-0-13-044911-5.